

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



LỚP: CS112.Q11.KHTN

MÔN HỌC: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

Luyện Tập Thiết Kế Thuật Toán
Nhóm 13

Nhóm 4:
Lê Văn Thức
Bảo Quý Định Tân

Giảng viên:
Nguyễn Thanh Sơn

1 Giới thiệu bài toán

Bài toán yêu cầu chúng ta thu thập đủ W ngôi sao từ N cấp độ. Mỗi cấp độ có hai mốc: 1 sao (chi phí a_i) và 2 sao (chi phí b_i , với $a_i < b_i$). Mỗi cấp độ chỉ được chọn một trạng thái duy nhất (0 sao, 1 sao hoặc 2 sao). Mục tiêu là tìm tổng chi phí thời gian nhỏ nhất để đạt được đúng W sao.

2 Phương pháp thiết kế thuật toán

2.1 Phương pháp sử dụng: Tham lam (Greedy Algorithm)

Để giải quyết bài toán này, chúng tôi sử dụng phương pháp thiết kế thuật toán Tham lam (Greedy) và phương pháp thiết kế Transform and Conquer với cấu trúc dữ liệu Hàng đợi ưu tiên (Priority Queue/Heap).

Cơ chế hoạt động: Bài toán yêu cầu đạt W sao. Thay vì cố gắng tìm tổ hợp tối ưu ngay lập tức, chúng ta sẽ thực hiện W bước. Tại mỗi bước, ta tìm cách *rẻ nhất* để tăng thêm đúng 1 ngôi sao vào tổng số sao hiện có.

Để tăng thêm 1 sao, ta xét 4 khả năng chuyển đổi trạng thái của các cấp độ:

1. **Thêm mới 1 sao ($0 \rightarrow 1$):** Chọn một cấp độ chưa chơi và hoàn thành mốc 1 sao. Chi phí: a_i .
2. **Nâng cấp ($1 \rightarrow 2$):** Chọn một cấp độ đang có 1 sao và hoàn thành mốc 2 sao. Chi phí: $b_i - a_i$.
3. **Thay thế loại 1 ($0 \rightarrow 2$ và $2 \rightarrow 1$):** Chọn một cấp độ mới lên thẳng 2 sao, đồng thời giảm một cấp độ khác từ 2 sao xuống 1 sao. Chi phí thực tế: $b_{new} - (b_{old} - a_{old})$.
4. **Thay thế loại 2 ($0 \rightarrow 2$ và $1 \rightarrow 0$):** Chọn một cấp độ mới lên thẳng 2 sao, đồng thời bỏ một cấp độ đang có 1 sao về 0 sao. Chi phí thực tế: $b_{new} - a_{old}$.

Để quản lý và truy xuất giá trị nhỏ nhất cho các thao tác trên một cách hiệu quả, thuật toán sử dụng 5 Min-Heaps:

- `pq_0_a`: Lưu chi phí a_i (cho thao tác $0 \rightarrow 1$).
- `pq_1_d`: Lưu chi phí $b_i - a_i$ (cho thao tác $1 \rightarrow 2$).
- `pq_0_b`: Lưu chi phí b_i (dùng cho thao tác thay thế, phần thêm vào).
- `pq_2_neg_d`: Lưu giá trị hoàn lại $-(b_i - a_i)$ (dùng cho thao tác giảm $2 \rightarrow 1$).
- `pq_1_neg_a`: Lưu giá trị hoàn lại $-a_i$ (dùng cho thao tác giảm $1 \rightarrow 0$).

2.2 Lý do lựa chọn phương pháp (Tính phù hợp)

1. **Cấu trúc tối ưu cục bộ (Optimal Substructure):** Bài toán có tính chất của một bài toán tối ưu hóa lồi (convex optimization) trên tập rời rạc. Việc chọn phương án chi phí biên (marginal cost) thấp nhất ở mỗi bước đảm bảo rằng sau W bước, tổng chi phí là nhỏ nhất. Không cần thiết phải duyệt lại các trạng thái quá khứ như Quy hoạch động (Dynamic Programming - DP).

2. **Không gian trạng thái quá lớn cho DP:** Nếu sử dụng DP dạng Balo (Knapsack), trạng thái sẽ là $DP[i][j]$ (xét đến gói i , đã có j sao). Với $N, W \leq 3 \cdot 10^5$, độ phức tạp $O(N \cdot W)$ là quá lớn cho giới hạn thời gian 5 giây, đồng thời mảng 2 chiều cũng gây tràn bộ nhớ (Memory Limit Exceeded).
3. **Khả năng mở rộng và sửa lỗi (Undo):** Điểm đặc biệt của thuật toán này là khả năng "hồi hận" thông qua các thao tác thay thế (Op 3 và Op 4). Nếu ở bước trước ta lỡ chọn một cấp độ giá rẻ nhưng nâng cấp lên 2 sao lại đắt, các thao tác thay thế cho phép ta đảo ngược quyết định đó để đạt cấu hình tối ưu hơn. Điều này chứng minh tính đúng đắn của chiến lược tham lam trong bài toán này.

3 Phân tích độ phức tạp

3.1 Độ phức tạp thời gian (Time Complexity)

Giả sử N là số lượng cấp độ và W là số sao cần đạt được.

- **Khởi tạo:** Ta duyệt qua N phần tử để đẩy vào Heap ban đầu. Chi phí: $O(N \log N)$ (hoặc $O(N)$ nếu dùng heapify).
- **Vòng lặp chính:** Ta lặp lại W lần để lấy đủ sao.
 - Ta thực hiện các thao tác peek (xem phần tử đầu) và pop (lấy phần tử đầu) trên các Priority Queue.
 - Do sử dụng cơ chế "Lazy Deletion" (chỉ loại bỏ phần tử rác khi nó trôi lên đầu heap), số lượng phần tử trong Heap có thể lớn hơn N nhưng bị giới hạn bởi tổng số thao tác đẩy vào.
 - Mỗi thao tác Heap (push/pop) tốn $O(\log M)$, với M là số phần tử trong Heap. Trong trường hợp tệ nhất, $M \approx N + W$.

Tổng độ phức tạp thời gian: $O((N + W) \log N)$. Với $N, W \leq 3 \cdot 10^5$, thuật toán hoàn toàn thỏa mãn giới hạn thời gian 5 giây (thực tế chạy khoảng 0.5s - 1s với Python + PyPy).

3.2 Độ phức tạp không gian (Space Complexity)

Đây là phần quan trọng đã được tối ưu hóa trong mã nguồn sử dụng:

- **Vấn đề của cài đặt thông thường:** Nếu lưu trữ Heap dưới dạng tuple '(value, index)' trong Python, mỗi phần tử là một object riêng biệt, gây tiêu tốn bộ nhớ rất lớn (overhead) và làm chậm tốc độ (cache miss).
- **Kỹ thuật tối ưu (Bitwise Packing):** Thuật toán sử dụng kỹ thuật nén dữ liệu:

$$\text{packed} = ((\text{value} + \text{OFFSET}) \ll 20) | \text{index}$$

Điều này giúp gói gọn cả giá trị chi phí và chỉ số mảng vào trong một số nguyên duy nhất (int 64-bit).

- **Không gian sử dụng:** Ta cần 5 Heaps và các mảng phụ trợ (a, b, state) . Tổng dung lượng bộ nhớ là tuyến tính $O(N + W)$.